

Manuscript Compiler for Obsidian

An Obsidian vault becomes a print-ready book — and the typesetting is done without a language model.

Elizabeth Sobiya · built on SuperDocs · Round 2, assigned build

RESULTS, MEASURED

458 ms	to compile 60 chapters — ~90,000 words, ~300 printed pages, 120 figures, 180 footnotes — with numbering continuous and correct across every chapter boundary
10 of 11	chapters come through byte-identical after editing the eleventh. Reordering the book re-renders zero
113 tests	pass on a fresh clone with no API key, no network, no vault and no Obsidian
20 · 6 · 7	real Word page breaks, real Word footnotes and embedded images, verified inside the exported .docx
4 of 500	SuperDocs operations for the entire project, including every live export and two full four-call round trips. Compiling costs zero. Exporting costs zero.

What it is, and who it is for

An author, a thesis writer or a documentation lead keeps a book as a few hundred notes. They can search it, link it and think in it — but they cannot *hand it to anyone*, because a vault is not a manuscript.

This closes that gap. One index note, the **spine**, lists the chapters in order. Compiling turns the vault into a real long-form document: chapters in order, heading levels normalised, front and back matter generated, inline notes converted to numbered footnotes, figures captioned and numbered continuously, citations resolved against a BibTeX file, and a table of contents built. Out come a print-ready **PDF** and a **DOCX** an **editor can mark up**, rendered by SuperDocs. It ships as an Obsidian plugin and as a headless CLI over the same compiler, so a book kept in git compiles in CI exactly as it does in the app.

The trade-off that decides everything

Typesetting is not a job for a language model. Chapter order, figure numbering, footnote numbering, citation resolution and heading normalisation are deterministic problems with one right answer. Handing them to a model makes them slower, dearer, and — worst — *non-repeatable*: compile twice, get two books.

So the compiler is pure, local, offline and free. SuperDocs does the two things it is genuinely better at: rendering a real Word and PDF file, and running **optional editorial passes** — draft a preface, write a jacket blurb, report whether chapter nine's voice has drifted. Each is opt-in, scoped to a slice rather than 300 pages, priced before it runs, and reviewed change by change. The cost of this split is that the interesting AI work sits at the edges rather than the centre. What it buys is an author who can recompile fifty times in an afternoon for nothing, and get the same book every time.

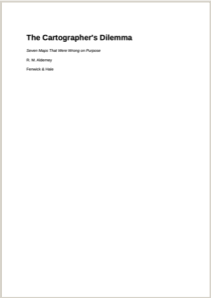
Two smaller trade-offs, made the same way. Numbers are assigned at *assembly*, not at render, so a chapter's bytes depend on that chapter alone — one extra pass, in exchange for a recompile that can name exactly which chapters moved, and prove it. And every id is keyed off a note's *path* rather than its position, which yields uglier ids and reordering that re-renders nothing at all.

What it will not do

It never invents anything. An unresolved citation prints a visible marker and fails the compile; a cross-reference to a figure no chapter places is left visible rather than given a plausible number; a missing figure is named, not skipped. A success message only ever means the file is on disk with bytes in it.

Honest gaps: the table of contents has **no page numbers** — pagination does not exist until the renderer lays the book out, and a fabricated number is worse than none. Footnotes land at the page foot in DOCX but are collected as endnotes in PDF; that is the export pipeline's behaviour, and it is reported as a bug. The prompt-injection scanner is a tripwire, not a classifier — the real protection is that no model-proposed change reaches the vault without a human approving it, item by item.

The output, unretouched



Title page, generated from the spine's frontmatter



Contents — clickable, indented, no faked page numbers



A chapter: numbered figure, footnotes, live citation

Every page here is output from **npm run demo:export**, unedited.

- **25-page PDF** · 24 bookmarks · 36 working hyperlinks
- **48 KB DOCX** · 20 real Word page breaks · 6 real Word footnotes · 7 embedded images
- **0 operations** for all of it — the compiler writes the markup itself rather than asking a model to